CAMERA CALIBRATION

ver 2

체커보드 캘리브레이션 결과

격자 크기 오설정을 바로잡고, 가장자리 뷰를 보강해 30장으로 마무리한 기록

Daheng MER2-240-159U3C · 2048 × 1200 · 9 × 8 inner corners · 125.0 mm

2026-08-18 · results/calibration.yaml · RMS 0.3518 px (30 views)

결론

캘리브레이션 완료

results/calibration.yaml — 이 값을 그대로 쓰면 된다

항목	값
fx, fy	2079.303, 2079.027 px
cx, cy	1033.235, 587.120 px (화면 중심 1024.0, 600.0)
왜곡 k1 k2 p1 p2 k3	−0.02827 +0.20515 −0.00343 +0.00140 −0.12017
RMS 재투영 오차	0.3518 px (30 views)
초점거리	7.17 mm @ 3.45 µm pixels
FOV	52.4° × 32.2°
보드	9 × 8 inner corners, 125.0 mm (실측)

신뢰 범위

화면 중심에서 r ≈ 0.9까지. 그 바깥은 관측된 코너가 전체의 0.5 %뿐이라 외삽이다.

검출

36장 촬영, 36장 전부 검출, 6장 프루닝 후 30장 사용.

판정 기준: RMS < 0.5 px 양호 · 0.5~1.0 px 이상치 확인 · > 1.0 px 구조적 문제

다만 RMS는 정확도가 아니다 — 7페이지 참조.

정정

ver 1의 진단은 틀렸다

세 번의 실패는 보드 탓이 아니라 격자 크기 하나였다

시도	결과	ver 1의 진단	실제 원인
1차	RMS 24.52 px	격자를 9×6으로 오설정	맞음 — 격자 크기
2차	RMS 1.99 px	보드가 평평하지 않음 (종이 주름)	틀림 – 역시 격자 크기 (10×9 설정, 실제 9×8)
3차	촬영 0장	테이프가 패턴을 가림	해당 없음 — 재촬영 자체가 불필요했다
현재	RMS 0.35 px	—	같은 사진으로 해결

보드는 처음부터 멀쩡했다

ver 1은 보드 재제작과 5 m 재촬영을 결론으로 냈지만, 둘 다 필요 없었다. 8/17에 찍어둔 사진 21장을 올바른 격자 크기로 다시 풀자 21장 전부 검출되고 RMS가 1.99 → 0.563 px로 떨어졌다.

근거

격자 크기를 고치자 전부 풀렸다

같은 사진 21장, 설정만 바꿔 재계산

설정	검출된 뷰	RMS	주점 (cx, cy)
10 × 9 (ver 1)	11 / 21	1.99 px	(1049.9, 557.4)
9 × 8 (실제)	21 / 21	0.563 px	(1030.2, 600.9)

주점이 답을 말해준다

화면 중심은 (1024, 600). 고친 쪽은 6 px 안에 들어오고, 틀린 쪽은 43 px 벗어나 있다.

독립 검증: detect_board_size.py captures/calib_003.png → 9 × 8 inner corners (10 × 9 squares)

왜 조용히 실패하나

OpenCV는 "격자가 없다"고 알려주지 않는다.

설정한 크기에 맞는 부분격자를 찾아내고, 사진마다 다른 위치를 잡는다.

검출은 매번 성공하고 프리뷰에도 코너가 예쁘게 찍힌다.

이상이 드러나는 것은 계산 단계뿐이다.

→ 촬영 전 5초면 확인된다.

보드

9 × 8 내부 코너, 125 mm 실측

board.json의 cols/rows는 칸 수가 아니라 내부 코너 개수다

내부 코너　9 × 8
촬영 시 방향 기준 — 가로 9개, 세로 8개

칸　10 × 9
내부 코너 n개 사이의 칸은 n − 1개

칸 크기　125.0 mm
실측값. 인쇄 배율 오차까지 반영된 값이다

패턴 크기　1250 × 1125 mm
흰 여백은 포함하지 않는다

재는 법 — 보드 바깥 테두리가 아니다
가장 바깥 내부 코너 → 반대편 가장 바깥 내부 코너
긴 축 8칸 (약 1000 mm)을 한 번에 재서 ÷ 8
짧은 축 7칸 (약 875 mm)을 재서 ÷ 7 로 교차검증
대각선은 재지 않는다 — 정사각 가정이 섞인다

square_size_mm은 K와 왜곡계수를 바꾸지 않는다
거리(tvec)와 mm 환산에만 들어간다. 단, 차량 외부 파라미터에서 1 % 오차 = 모든 거리 1 % 오차.

검산
125 mm로 계산한 30장의 카메라–보드 거리가 2.34 ~ 4.46 m. 실제 촬영 범위와 일치한다.

프루닝

36장 중 6장을 버렸다

python calibrate.py --prune 1.0

뷰	오차	기울기	선명도	버린 이유
calib_023	5.96 px	41.1°	276	흔들림
calib_032	2.11 px	55.8°	242	흔들림 + 극단 각도
calib_022	1.76 px	11.3°	160	흔들림
calib_008	1.23 px	16.8°	306	중앙 · 여분
calib_019	1.20 px	10.7°	389	중앙 · 여분
calib_012	1.19 px	14.1°	226	중앙 · 여분

선명도 중앙값

426

이 값을 크게 밑도는 뷰가 흔들린 뷰다.

가장자리 뷰일수록 흔들린다

버린 6장 중 3장은 새로 찍은 가장자리·큰 각도 뷰다. 화면 끝까지 보드를 밀어 넣고 크게 기울인 자세는 손으로 버티기 어렵다.
촬영 당시 gain이 16 dB였던 것이 직접적인 원인 — 노출을 늘리지 말고 조명을 더해야 한다.

함정

RMS는 올라갔는데 결과는 좋아졌다

k3를 푼 모델과 0으로 묶은 모델이 같은 픽셀을 어디로 보내는가 — 데이터가 모델을 붙들고 있는 정도

반경	17장 (가장자리 부족)	30장 (가장자리 보강)
r 0.2	0.02 px	0.01 px
r 0.6	0.09 px	0.06 px
r 0.8	0.18 px	0.07 px
r 0.9	0.83 px	0.28 px
RMS	0.3139 px	0.3518 px

읽는 법

두 모델이 갈라지는 곳이 곧 데이터가 없는 곳이다.

r 0.9에서 0.83 → 0.28 px. 믿을 수 있는 범위가 세 배 넓어졌다.

RMS만 보면 17장이 낫다. 그러나 17장 쪽은 화면 가장자리를 추측하고 있었을 뿐이다.

k3도 −0.271 → −0.120으로 양전해졌다. 가장자리 데이터가 들어오면서 계수가 실제로 구속되기 시작한 것이다.

커버리지

가장자리를 채운 효과

왜곡계수는 거의 전적으로 화면 가장자리 코너가 결정한다

중심에서의 거리	21장	36장
0.0 – 0.2	30.6 %	24.5 %
0.2 – 0.4	48.7 %	44.0 %
0.4 – 0.6	18.1 %	22.8 %
0.6 – 0.8	2.5 %	7.5 %
0.8 – 1.0	0.1 % (2개)	1.2 % (31개)
최대 반경	0.87	0.96
3 × 3 셀	5 / 9	7 / 9

가운데만 찍으면 RMS는 낮게 나오지만 화면 가장자리에서 틀린다

capture_board.py의 HUD는 셀이 7칸 미만이면 종료 시 경고를 낸다.

1.0 = 화면 모서리

21장은 r 0.87까지밖에 닿지 않았고, 그 바깥엔 코너가 2개뿐이었다.

남은 빈 칸

3×3 중 좌상·우상 두 곳. 채우려면 조명을 올리고 선명한 뷰 5장 정도.

절차

재현 순서

격자 크기 확인이 항상 먼저다

```
python dhcam.py
    카메라 연결 확인. gain이 16 dB 근처면 조명을 더한다.
python detect_board_size.py --write
    격자 크기 확정. 5초. 이 단계를 건너뛴 것이 실패 원인이었다.
python snap.py
    한 장 찍어 검출되는지 확인. 실패하면 여기서 멈춘다.
python capture_board.py
    셀 9/9, 기울기 20~40°를 절반. 화면 네 귀퉁이를 반드시 채운다.
python calibrate.py --prune 1.0
    뷰별 오차 표를 반드시 읽는다.
```

captures/ 를 비우고 시작한다

다시 실행하면 기존 사진에 덧붙는다. results/undistorted/ 도 비워지지 않아 이전 결과가 섞인다.

RMS	조치
< 0.5 px	사용 가능
0.5 ~ 1.0 px	이상치 확인 후 prune
> 1.0 px	격자 크기부터 의심

촬영 전 고정

초점과 조리개를 운용 상태로 맞추고 잠근다. 초점을 바꾸면 K가 변한다.

사용

결과를 코드에서 쓰기

results/calibration.yaml — OpenCV FileStorage

```
import cv2, numpy as np

fs = cv2.FileStorage("results/calibration.yaml", cv2.FILE_STORAGE_READ)
K  = fs.getNode("camera_matrix").mat()
D  = fs.getNode("distortion_coefficients").mat()
fs.release()

undistorted = cv2.undistort(frame, K, D)
```

함께 저장되는 것

calibration.json — 뷰별 오차와 메타데이터

results/undistorted/ — 좌측 원본 / 우측 보정 비교 이미지

자율주행으로 넘어갈 때

- r 0.9 바깥은 외삽이다 — 화면 네 귀퉁이를 쓰는 처리라면 그만큼 감안한다
- 차량 장착 후 진동·온도로 값이 틀어진다. 재검증을 일정에 넣는다

정리

체크리스트

- ☐ 촬영 전 detect_board_size.py 로 격자 크기 확정
- ☐ 초점·조리개를 운용 상태로 고정하고 잠금
- ☐ captures/ 비우고 시작
- ☐ 기울기 20~40° 뷰를 절반 정도
- ☐ RMS가 아니라 가장자리 커버리지로 품질을 판단
- ☐ 칸 크기는 코너→코너 전체를 재서 칸 수로 나눈 실측값
- ☐ gain 16 dB 근처면 조명 보강 (노출 상향 아님)
- ☐ 3×3 셀 9칸 전부, 화면 네 귀퉁이 포함
- ☐ 뷰별 오차 표 확인 후 --prune
- ☐ 차량 장착 후 재검증을 일정에 포함

한 줄 요약 — 격자 크기를 먼저 확정하고, 화면 가장자리를 채우고, RMS 하나로 판단하지 않는다.